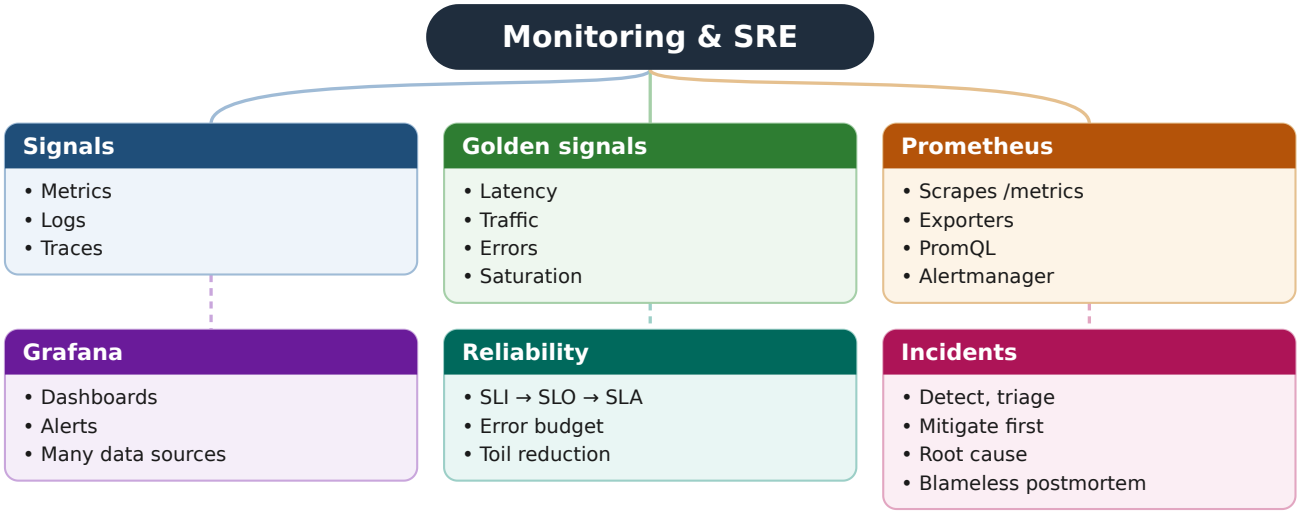
13. Monitoring & Observability

In this section: 13.1 Concepts · 13.2 Prometheus · 13.3 Grafana · 13.4 Logging and tracing · 13.5 Troubleshooting with monitoring · 13.6 SRE practices · 13.7 Incident management



13.1 Concepts

Term	Meaning
Monitoring	Watching known signals and alerting when something is wrong
Observability	Being able to understand **why** a system behaves a certain way from its outputs
Metrics	Numbers over time (CPU %, requests/sec, error rate)
Logs	Timestamped text records of events
Traces	The path of one request through many services, with timing per step
Alert	A notification when a rule is broken (sent to email, Slack, PagerDuty)
Dashboard	Visual panels of metrics
SLI	Service Level Indicator: what you measure (e.g. % of successful requests)
SLO	Service Level Objective: your target (e.g. 99.9% success over 30 days)
SLA	A promise to customers, with penalties
Error budget	Allowed failure = 100% − SLO (99.9% → about 43 minutes per month)

Golden signals: latency, traffic, errors, saturation. **USE method (resources):** Utilization, Saturation, Errors. **RED method (services):** Rate, Errors, Duration.

13.2 Prometheus

- **Pull model:** Prometheus scrapes `/metrics` endpoints on a schedule and stores time series.

- **Exporters** expose metrics for things that don't have them: node_exporter (Linux), blackbox_exporter (HTTP checks), database exporters.
- **Alertmanager** groups alerts and sends notifications.
- On Kubernetes, install everything with the **kube-prometheus-stack** Helm chart (Prometheus, Alertmanager, Grafana, exporters).

Metric type	Meaning
Counter	Only goes up (total requests) — use `rate()`
Gauge	Goes up and down (memory in use)
Histogram	Buckets of values (request durations) — use for percentiles

PromQL examples:

```
up == 0                                                  # targets that are down
rate(http_requests_total[5m])                            # requests per second
sum(rate(http_requests_total{status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m]))                   # error ratio
# p95 latency (95% of requests are faster than this)
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)                    # CPU %
```

Alert rule:

```
groups:
  - name: app
    rules:
      - alert: HighErrorRate
        expr: |
          sum(rate(http_requests_total{status=~"5.."}[5m]))
            / sum(rate(http_requests_total[5m])) > 0.05
        for: 5m
        labels: { severity: critical }
        annotations:
          summary: "More than 5% of requests are failing"
```

13.3 Grafana

- Connects to data sources (Prometheus, Loki, CloudWatch) and builds dashboards and alerts.
- Import community dashboards (e.g. Node Exporter Full) and build your own app dashboard: request rate, error rate, latency, CPU/memory.
- Port-forward on K8s: `kubectl port-forward svc/<grafana-svc> 3000:80 -n monitoring`. Admin password comes from a Kubernetes Secret.

13.4 Logging and tracing

Tool	Use
Loki + Promtail / Grafana Alloy	Collect and search logs, viewed in Grafana
ELK / OpenSearch	Elasticsearch/OpenSearch + Logstash + Kibana: log search
CloudWatch Logs + Logs Insights	AWS central logs and queries
OpenTelemetry	Standard way to produce metrics, logs and traces from apps
Jaeger / Tempo / AWS X-Ray	Store and view traces

Good logs: structured (JSON), with timestamp, level, request ID, no secrets or personal data.

13.5 Troubleshooting with monitoring

1. Alert fires → open the dashboard. **What changed and when?** (deploy, traffic spike, dependency down)
2. Golden signals: errors up? latency up? traffic up? saturation (CPU, memory, disk, connections)?
3. Narrow it down: which service, which endpoint, which node/pod?
4. Logs for that time and component → find the error.
5. Traces → which step is slow.
6. Mitigate first (roll back, scale up), then find the root cause, then write a short post-incident review.

13.6 SRE practices

Practice	Meaning
SLOs and error budgets	Reliability target agreed with the business; if the budget is used up, slow down releases and fix reliability
Toil reduction	Automate repetitive manual work (SREs aim to keep toil below ~50% of their time)
Alert on symptoms	Alert on what users feel (errors, latency), not on every CPU spike. Every alert must be actionable
Runbooks	Step-by-step instructions linked from each alert
Capacity planning	Forecast growth; load test before big events
Blameless postmortems	After an incident, focus on what failed in the system and process, not on who
Chaos engineering	Break things on purpose in a controlled way to find weaknesses

13.7 Incident management

Step	What happens
Detect	Alert fires or a user reports a problem
Triage	Decide severity (SEV1 = major outage … SEV4 = minor) and assign an incident commander
Communicate	Status updates to stakeholders on a fixed schedule
Mitigate	Restore service first: roll back, fail over, scale up, disable a feature
Resolve	Find and fix the root cause
Learn	Postmortem: timeline, impact, root cause, action items with owners

Key metrics: MTTD (time to detect), MTTR (time to recover), change failure rate, deployment frequency, lead time for changes (the last four are the **DORA metrics**).

REMEMBER

- **SLI** = what you measure · **SLO** = your target · **SLA** = promise to customers.
- Alert on what users feel; every alert must be actionable.
- During an incident: **restore service first, investigate second**.